EWIS API — User Attribution & Audit Trail Design

Replacing database-account identity with application-level (Entra) attribution, without disturbing the legacy system

Prepared for: EWIS Platform / Data Governance • Status: Design proposal • Applies to: EWIS API (dynamic Oracle OData service)

---

1. Executive Summary

The legacy EWIS application identified users by giving **each person their own Oracle database account**, and its audit tables record "who did what" by reading the Oracle session user (`USER`). The modern EWIS API instead authenticates people through **Microsoft Entra** and connects to Oracle with a **single shared service account**. That is the correct, more secure architecture — but it means the legacy audit triggers can no longer see the real person; every change now records the shared account name.

This document proposes a small, **additive** enhancement: an application-level audit trail — a single consolidated table, `EWIS_API_AUDIT`, populated by the API with the real Entra user for every create, update, and delete. It does **not** modify any legacy table, trigger, or package. It is also easier to read than the legacy audit: one row per change instead of many rows spread across dozens of per-table shadow tables.

2. Background — The Legacy Approach Put Users Inside the Database

In the legacy system, application users were provisioned as **real Oracle database accounts**. When a person signed in, their identity *was* their Oracle login, so the database's built-in `USER` value equalled the person (e.g. `CALINN`, `TDMERKEY`). Every "who" stamp in the system flows from that single fact.

> **Why user-accounts-in-the-database is a recognized anti-pattern:**
>
> - **Large attack surface.** Every employee is a credential directly on the production database — far more exposure than a single application account behind a service tier.
> - **No central identity / SSO.** Identity is bound to the database engine, not to the organization's directory. It cannot participate in Entra, MFA, conditional access, or centralized joiner/mover/leaver processes.
> - **Operational burden & offboarding risk.** Accounts, passwords, and privileges must be created, rotated, and revoked per person, per environment. A missed revocation leaves a live database credential.
> - **Privilege sprawl.** Direct database accounts can be used outside the application, making least-privilege and separation-of-duties hard to enforce.
> - **Licensing & scaling friction.** Named database users tie user growth to database administration rather than to an identity provider.

The modern practice — and what the new API does — is to keep **people out of the database entirely**: authenticate them in Entra, and let the application connect to Oracle through one managed service account. The trade-off is that the database can no longer identify the person on its own; the application must carry that identity. That is exactly the gap this design fills.

3. How Legacy Auditing Works Today

Confirmed by directly reading the Dev database's trigger and package source. For a base table such as `EC_LABLOGS`:

Mechanism	Behavior
`EC_LABL_WHO_TBIUR` (trigger)	On insert/update sets `CREATE_USER` / `MODIFY_USER := USER` (the Oracle session user).
`EC_LABL_TBIUR` (trigger)	Sets numeric `CREATED_BY` / `LAST_UPDATED_BY` via `ec_api_persons.get_user_id(USER)`.
`EC_LABL_AUDIT_TBUDR` (trigger)	On update/delete calls package `EC_APP_AUDIT` to write shadow-table rows.
`EC_LABLOGS_A` (shadow table)	Column-level history: one row *per changed column* (old value, new value, event). Its own actor column is again stamped from `USER`.

Two facts matter for this design: (1) **every identity stamp resolves to Oracle `USER`** — nothing reads a client identifier or any passed-in value; and (2) the audit is **per-table and column-level**. There is a separate `*_A` shadow table for each base table, and a single logical edit fans out into many audit rows.

Why the legacy audit is hard to read

To answer "what did Chris change on lab-log entry 203741, and when?" you must:

1. Know that `EC_LABLOGS` is audited by `EC_LABLOGS_A` (a different table name per entity).
2. Filter that table by the row's key and timestamp.
3. Read *several* rows — one per changed column — and mentally reassemble them into a single edit.
4. Read the actor from a separate who-column, and interpret inconsistent event labels (`UPDATING`, `DELETE`).

A cross-entity question — "everything user X changed today" — is worse: it requires querying and unioning **dozens of different `*_A` tables**. The new design answers both questions from one table with one row per change (see §7).

4. What Changed — and Why Attribution Broke

The API connects to Oracle as the shared account `ENVR`. A live session check confirms the session presents as `SESSION_USER = ENVR` with `CLIENT_IDENTIFIER = NULL`. Because the legacy triggers stamp `USER`, **every**

write through the API records `ENVR` instead of the real person — and since the "who" triggers overwrite the column unconditionally, the application cannot fix this by passing a value into the existing columns. The real user is an Entra identity that, correctly, **is not a database user**. Legacy attribution therefore cannot represent them, and a new trail is required.

5. The New Method — Application-Level Audit Trail

The API records the real Entra user for each mutating operation into one dedicated table. Key properties:

- **Identity from Entra (JWT).** The signed-in user's `oid` and display name are read from the access-token claims the GUI already sends — no extra call to Entra. An optional HTTP header provides a fallback.
- **Single consolidated table.** All entities, all operations, land in `EWIS_API_AUDIT`.
- **One row per change,** with the details captured as JSON (`EVENT_JSON`).
- **Best-effort.** If the audit write fails, the underlying data change is never blocked.
- **Self-provisioning.** The table is auto-created on startup if absent (configurable), so each environment sets itself up.

> **Non-impact — it enhances, it does not disturb.**
>
> - **NO** changes to any legacy table, trigger, sequence, or the `EC_APP_AUDIT` package.
> - **NO** change to how existing writes behave; the legacy `*_A` trail keeps working exactly as today.
> - **ADDS** a real human actor (not `ENVR`) on every API change.
> - **ADDS** a single, easy-to-query history across all entities.
> - **SAFE** if identity or the audit table is unavailable, data operations continue uninterrupted.

6. Owner Resolution

The API determines the acting user in this order:

1. **JWT claim (primary):** `oid` → `OWNER_USER`, `name` → `OWNER_DISPLAY`, read from the validated Entra token.
2. **Header (fallback):** `X-EWIS-On-Behalf-Of` and `X-EWIS-On-Behalf-Of-Name`.
3. **None:** the row is still written with a null owner (best-effort). A later switch, `Audit:RequireOwner=true`, can enforce attribution once the GUI is fully cut over.

7. Design Details

7.1 Audit table

```
EWIS_API_AUDIT
  AUDIT_ID       NUMBER        -- primary key, from sequence EWIS_API_AUDIT_SEQ
  ENTITY_NAME    VARCHAR2      -- e.g. EC_LABLOGS        (indexed query helper)
  OPERATION      VARCHAR2      -- INSERT | UPDATE | DELETE (indexed query helper)
  OWNER_USER     VARCHAR2      -- Entra oid            (JWT claim, header fallback)
  OWNER_DISPLAY  VARCHAR2      -- display name
  DB_USER        VARCHAR2      -- 'ENVR' (cross-reference to the legacy trail)
  AUDIT_TS_UTC   TIMESTAMP     -- server time, UTC
  EVENT_JSON     CLOB          -- the CRUD event: entity, operation, pk, new values
```

7.2 EVENT_JSON — examples for each operation

CREATE (POST)

```
{
  "entity": "EC_LABLOGS",
  "operation": "INSERT",
  "pk": { "ID": 204512 },
  "values": {
    "SHIP_ID": 52035,
    "ENTRYNO": "00",
    "SAMPLENO": "A-1",
    "NOTES": "Initial sample log"
  }
}
```

UPDATE (PATCH)

```
{
  "entity": "EC_LABLOGS",
  "operation": "UPDATE",
  "pk": { "ID": 203741 },
  "values": {
    "APPROVAL_DATETIME": "2026-07-
21T09:15:00",
    "USER_WHO_APPROVED": "CALINN"
  }
}
```

DELETE

```
{ "entity": "EC_LABLOGS", "operation": "DELETE", "pk": { "ID": 203741 } }
```

Note: the `values` object holds the new values supplied on the call. The INSERT `pk` is the sequence-assigned key, captured via a `RETURNING` clause (optional; if not enabled, `pk` is omitted on insert). Reads (GET) are not audited.

7.3 A stored row (UPDATE)

```
AUDIT_ID       1001
ENTITY_NAME    EC_LABLOGS
OPERATION      UPDATE
OWNER_USER     5b9e6f0a-3d21-4c8e-9f77-2c1a...   (Entra oid)
OWNER_DISPLAY  Chris Alinn
DB_USER        ENVR
AUDIT_TS_UTC   2026-07-21T15:15:03Z
EVENT_JSON     { ...the UPDATE example above... }
```

7.4 Readability — legacy vs. new, same edit

Question	Legacy (`*_A` shadow tables)	New (`LEWIS_API_AUDIT`)
One edit to entry 203741	Multiple rows in `EC_LABLOGS_A` (one per changed column); actor in a separate column; must reassemble.	One row; all changed fields in `EVENT_JSON`; actor in `OWNER_DISPLAY`.
Everything user X did today, all entities	Query and UNION dozens of different `*_A` tables.	One filter: `WHERE OWNER_USER = :x AND AUDIT_TS_UTC >= :today`.
Who is the actor?	Oracle `USER` — now always `ENVR` via the API.	The real Entra person (`oid` + display name).

7.5 Configuration

Setting	Default	Purpose
`Audit:Enabled`	true	Master on/off for the trail.
`Audit:AutoCreate`	true	Create the table/sequence at startup if missing (turn off where DBAs pre-create).
`Audit:RequireOwner`	false	When true, reject writes lacking an owner — flip after GUI cutover.

8. GUI Implementation Over Time

Because the owner is optional and the audit is best-effort, this can be adopted gradually with no coordinated release. The identity values are already present in the browser after sign-in (MSAL cache) — no extra Entra calls.

Phase	Action	Result
0 — Server first	Deploy the API with the audit trail. No GUI change.	Every change is audited. If the GUI already sends its Entra bearer token, the real user is captured immediately; otherwise the owner is null.
1 — One interceptor	Add a single HTTP interceptor in the GUI that attaches identity to all write calls: `oid = account.idTokenClaims.oid`, `name = account.name`.	All screens attribute correctly at once — no per-screen work.
2 — Enforce	Confirm coverage (owner-present rate), then set `Audit:RequireOwner=true`.	Attribution is guaranteed going forward.

> **GUI effort is minimal:** read two values from the cached MSAL account and attach two request headers (or simply ensure the existing bearer token is sent). No new libraries, no calls to Entra, no screen-by-screen rewrite required.

9. Summary

- The legacy system embedded users as database accounts and derived all attribution from Oracle `USER` — a security anti-pattern the new API deliberately moves away from.
- Moving to Entra + a shared service account is the right architecture, but it leaves the legacy audit unable to name the real person.
- The proposed application-level trail restores real attribution, is far easier to read (one row per change, one table for all entities), and is purely additive — the existing system is untouched and continues to function.
- It can be turned on server-side immediately and adopted in the GUI incrementally, then enforced when ready.
